USTHB - Faculté d'Informatique
4ème année ingénieurs IL

TD 02 Two Track Unified Process (2TUP)

Processus de développement logiciel

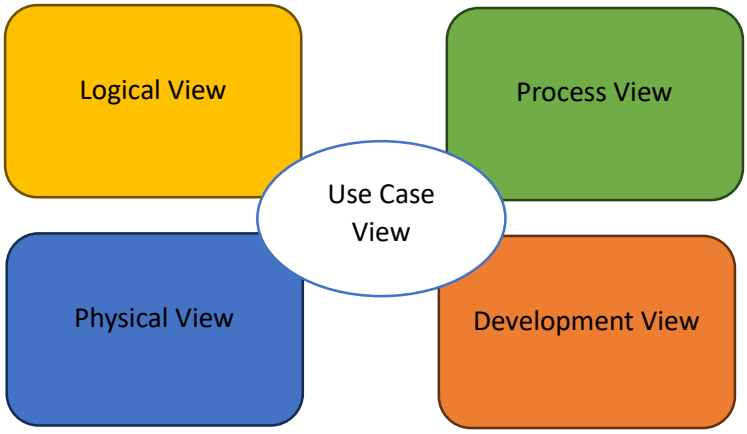
L'objet d'un processus de développement est de produire des logiciels de qualité qui répondent aux besoins de leurs utilisateurs dans des temps et des coûts prévisibles.

Processus Unifié (UP)

Le Processus Unifié (UP pour Unified Process) est un processus de développement itératif et incrémental.

Il s'agit d'une démarche s'appuyant sur la modélisation UML pour la description de l'architecture du logiciel (fonctionnelle, logicielle et physique) et la mise au point de cas d'utilisation permettant de décrire les besoins et exigences des utilisateurs. Il répond aux caractéristiques suivantes :

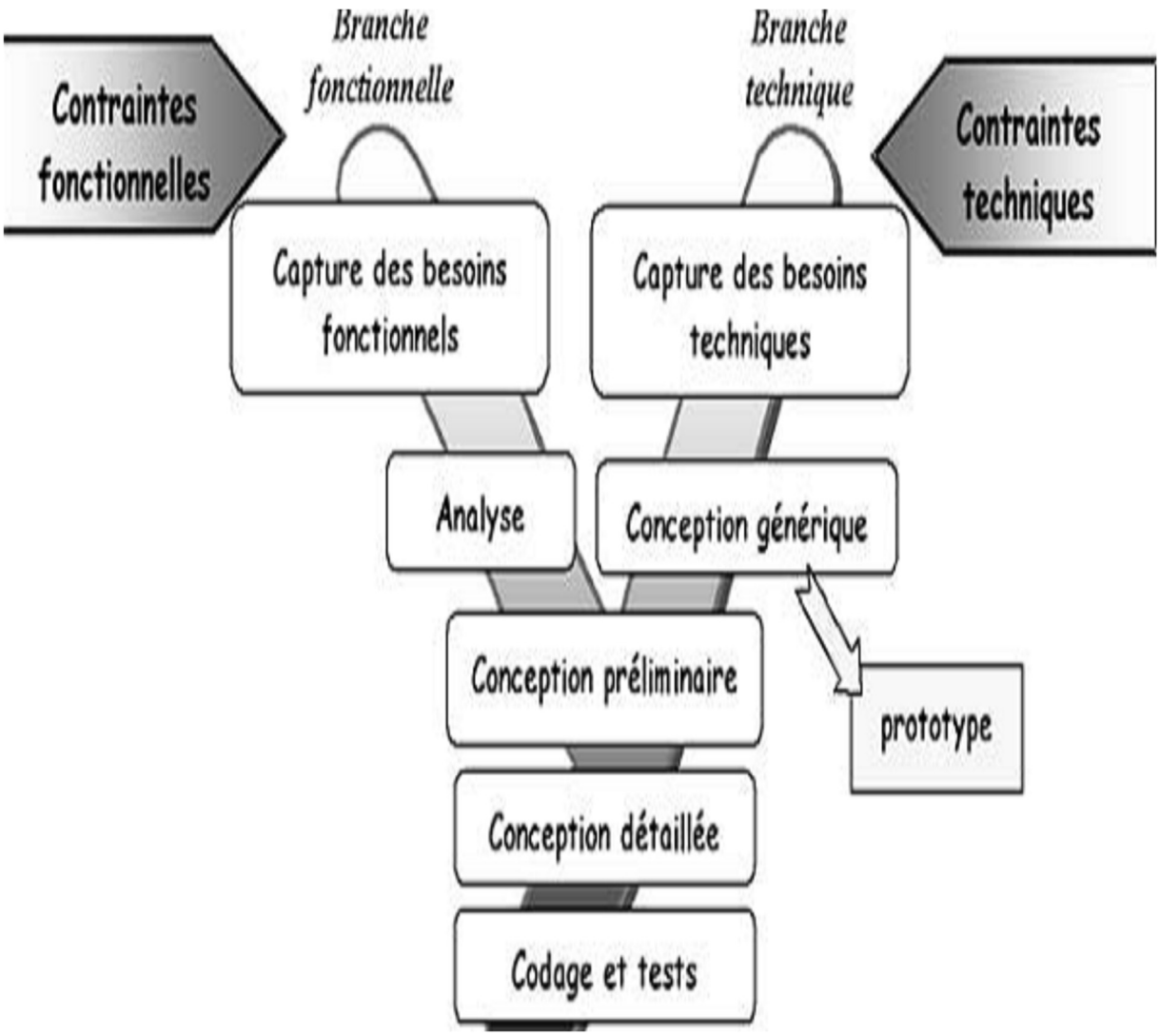
1. Itératif et incrémental
2. Centré sur l'architecture
3. Piloté par les cas d'utilisation



Méthode 2TUP

Créé par Valtech, 2TUP (2 tracks unified process) est un processus de développement logiciel qui implémente le Processus Unifié. « 2 Track » signifie littéralement que le processus suit deux chemins : « fonctionnels » et « d'architecture technique ».

Pourquoi 2tup ?



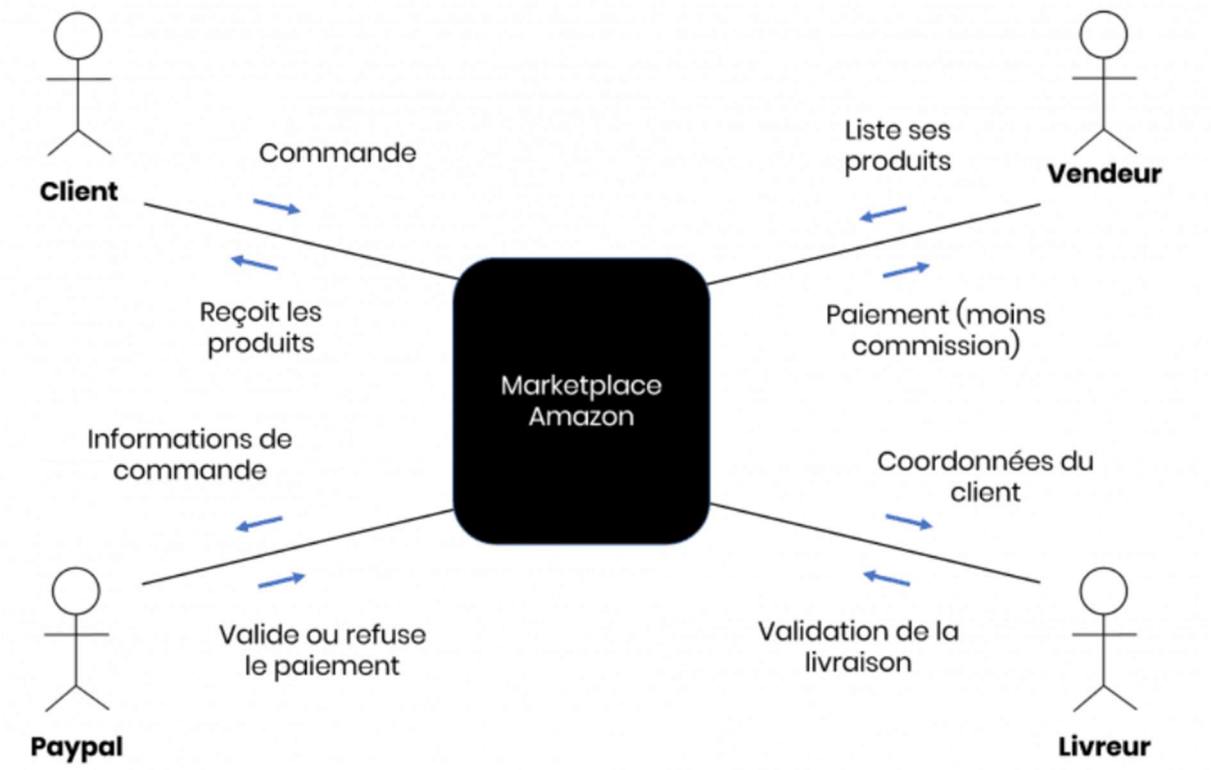
Capture initiale des besoins :

1. Interview pré étude ;

L'étude préliminaire (ou pré-étude) est la toute première étape de notre processus de développement. Elle consiste à effectuer un premier repérage des besoins fonctionnels et opérationnels, en utilisant principalement le texte, ou des diagrammes très simples.
Output : Cahier des charges préliminaires.

2. Modélisation du contexte :
   a. Identification des acteurs : Les acteurs candidats sont systématiquement : les utilisateurs humains directs : identifiez tous les profils possibles, sans oublier l'administrateur, l'opérateur de maintenance, etc. ; les autres systèmes connexes qui interagissent aussi directement avec les systèmes.
   b. Identifier les messages : Un message représente la spécification d'une communication unidirectionnelle entre objets qui transporte de l'information avec l'intention de déclencher une activité chez le récepteur.
   c. Modéliser le contexte : Tous les messages (système acteurs) identifiés peuvent être représentés de façon synthétique sur un diagramme, que l'on peut qualifier de diagramme de contexte dynamique. Le système étudié est représenté par un

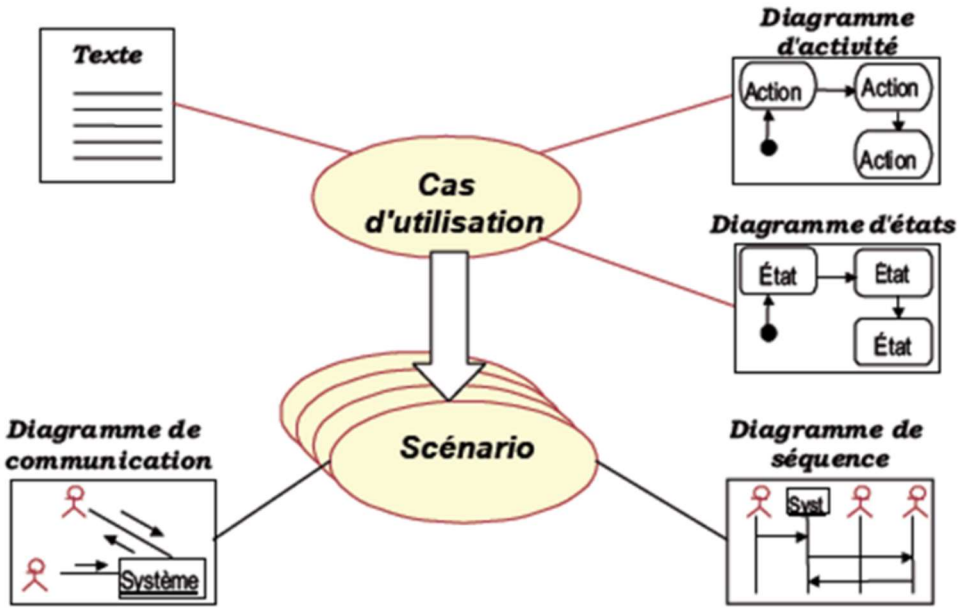
participant central, ce participant central est entouré par d'autres participants symbolisant les différents acteurs, des liens relient le système à chacun des acteurs et sur chaque lien sont montrés les messages en entrée et en sortie du système, sans numérotation.



Capture des besoins fonctionnels :

La capture des besoins fonctionnels est la première étape de la branche gauche du cycle en Y. Elle formalise et détaille ce qui a été ébauché au cours de l'étude préliminaire. Elle décrit les différentes façons qu'auront les acteurs d'utiliser le futur système.

1. Identifier les cas d'utilisation : Un cas d'utilisation n'est pas une transaction, ni une fonction. (output : DUC)
   a. Distinguer l'acteur principal des acteurs secondaires.
   b. Nommer les cas d'utilisation avec un verbe à l'infinitif suivi d'un complément.
   c. Etablir une première description succincte de chaque cas d'utilisation candidat. Détailler les rôles (principal ou secondaire) et le sens des associations.
2. Décrire les cas d'utilisation (output : fiches descriptives + diagrammes dynamiques simples)
   a. Utilisez le style de description adapté.
   b. Ne mélangez pas l'IHM et le fonctionnel.
   c. Compléter les descriptions textuelles avec des diagrammes dynamiques simples.



3. Organiser les cas d’utilisation (output : Packages de spec fonctionnelle)
   a. Les relations << INCLUDE>> entre cas d'utilisation.
   b. Inclusion : pas de décomposition fonctionnelle.
   c. Les relations << EXTEND > entre cas d'utilisation.
   d. Les relations de généralisation entre cas d'utilisation.
   e. Vous pouvez aussi généraliser les acteurs.
4. Identifier les classes candidates (output : diagramme de classes participantes)
   a. Les compléments des cas d’utilisation sont considérés comme des classes (exemple ajouter élève => élève = classe)
   b. Identification par responsabilité : Une responsabilité est une sorte de contrat, ou une obligation, pour une classe. Les attributs, les opérations, et les associations représentent les propriétés élémentaires qui contribueront à remplir les responsabilités de la classe. Exemple : (la classe véhicule implique : connaître ses capacités (poids max. en charge, encombrement, etc.) ; connaître son agence de rattachement ; savoir s'il est affecté à une mission, ou s'il est libre.)
   c. Déterminer les cardinalités.
5. Valider et consolider (output : DUC raffiné + traçabilité)

6. Il faut assurer la traçabilité des cas d'utilisation avec l'expression des besoins

Capture des besoins techniques :

1. Capture des spécifications techniques (output : Config materielle + style d'architecture)
2. Spécification logicielle initiale (input : UC fonctionnels, output : UC techniques)
3. Spécification logicielle détaillée (output : Dictionnaire des termes techniques + UC techniques détaillés + Couches logicielles)

Analyse

1. Découpage en catégories : Organiser les équipes d'analystes, puisqu'elles vont pouvoir travailler sur des ensembles cohérents et faiblement couplés assure l'évolutivité et la facilité de maintenance, et favoriser la réutilisation, en séparant notamment les parties applicatives des parties métiers. (stéréotyper les classes) (output : diagramme de classe préliminaire + dépendances entre catégories définies)
2. Développement du modèle statique (input : cahier des charges + fiches descriptives, output : diagramme de classe détaillé, complété, optimisé)
   a. Affiner les classes : Il faut éliminer les classes redondantes, les classes vagues, les classes à la place d'attribut, les classes à la place d'un rôle, les classes représentant des acteurs, les classes de conception, les classes représentant des groupes d'objets
   b. Affiner les associations : Eliminer les associations non structurelles : ce sont les liens structurels qui se caractérisent par une certaine durée et une certaine dynamique, et éliminer les associations redondantes : elles peuvent être retrouvées par navigation grâce aux associations existantes. Il faut utiliser aussi l'agrégation et la composition.

		Exigence 1	Exigence 2	Exigence 3	Exigence 4	Exigence 5
Use case A	scénario 1					
	scénario 2		X		X	X
	scénario 3		X		X	X
Use case B	scénario 1					
	scénario 2		X		X	
	scénario 3		X		X	
	scénario 4					
Use case C	scénario 1					
	scénario 2		X		X	

   c. Ajouter les attributs : éliminer les attributs redondants, distinguer les attributs dérivés (ex : âge dérivé de date de naissance et date actuelle), distinguer les attributs de classe
   d. Ajouter les opérations (éviter constructeurs, getters, setters)
   e. Optimiser avec la généralisation

3. Développement du modèle dynamique : activité itérative, fortement couplée avec l'activité de modélisation statique (input : DUC)
    a. Identifier les scenarios (output : liste des scénarios)
    b. Formaliser les scénarios : En analyse, un scénario représente un ensemble ordonné de messages échangés par des objets. On parle ici d'objet au sens large : instance de classe d'analyse ou instance d'acteur. (output : Diagramme d'interaction)
    c. Construire les diagrammes d'état
    d. Valider le modèle dynamique : valider les diagrammes d'états avec les diagrammes d'interactions
    e. Confronter les modèles statiques et dynamiques : Compléter les diagrammes de classes avec les attributs et opérations identifiées grâce à l'analyse dynamique

Conception Générique

La conception générique consiste à développer la solution qui répond aux spécifications techniques. L'architecte technique détermine et structure les besoins techniques suivant les couches du modèle de spécification logicielle. La conception générique permet de formaliser, sous la forme de classes techniques réutilisables, les règles de conception pour l'ensemble d'un système à développer.

1. Modèle logique (input : couches logicielles + UC techniques détaillés, output : mécanismes de modèles de conception de frameworks + modèle logique) : Les classes, les interfaces et les frameworks techniques représentent les briques de construction d'un modèle logique de conception générique. Le modèle logique est organisé par packages de classes qui représentent les frameworks développés pour résoudre les problèmes purement techniques. Le modèle logique est donc organisé suivant les dépendances qui s'établissent entre frameworks techniques.
2. Le modèle d'exploitation montre l'organisation des composants correspondant aux différents frameworks techniques. Il propose une vision statique de l'organisation des éléments physiques logiciels du système => élaboration le diagramme de composant (exécutables, librairies, fichier) correspondant aux frameworks techniques
3. Modèle de configuration logicielle : Organiser et préciser la configuration en indiquant les dépendances entre composants. Identifier aussi les sous-ensembles(sous-système) pouvant se fabriquer et s'utiliser indépendamment les uns des autres
4. Génération de code : Un générateur de code doit nous permettre d'accéder à toutes les informations du modèle UML et de produire des fichiers qui restituent cette information sous le format de votre choix. La conception générique consiste à développer le squelette technique d'un projet => Implémenter le framework technique
5. Prototypage : le prototype valide et prépare les trois niveaux de tests requis à la recette technique du système. Il inclut le test unitaire des composants techniques, il prépare les tests d'intégration dans le cadre de la préparation au

développement du système complet et il répond aux spécifications techniques. Le prototype valide et termine éventuellement l'étape de conception générique.

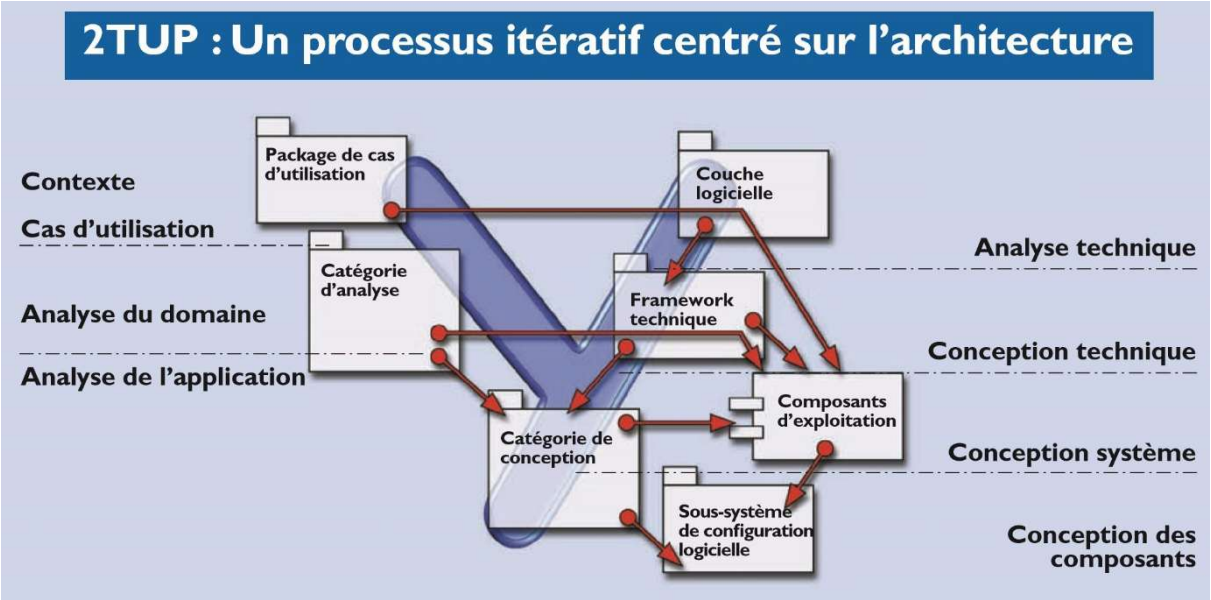
Conception Préliminaire

La conception préliminaire est avant tout affaire d'organisation ; elle s'appuie sur les points de vue de spécification fonctionnelle et structurelle de l'analyse, mais également sur les frameworks de la conception technique. Elle se termine lorsque la conception est organisée suivant son déploiement cible, son modèle d'exploitation, son modèle logique.

1. Concevoir le déploiement : Développement du modèle de déploiement : Le déploiement d'une solution client/serveur se construit sur la définition des postes de travail. Le poste de travail représente un ou plusieurs acteurs pouvant être localisés sur une machine (diagramme de déploiement)
2. Concevoir le modèle d'exploitation : le modèle d'exploitation va définir les applications installées sur les postes de travail, les composants métiers déployés sur les serveurs et les instances de bases de données implantées sur les serveurs également.
3. Organiser le modèle logique : Le modèle logique est précisément celui de la représentation des classes organisées en catégories. (modèle logique de conception)
4. Concevoir les interfaces : Dans un modèle, l'interface d'une catégorie se présente donc sous la forme des classes et des interfaces utilisables de l'extérieur de la catégorie. L'interface constitue le seul point d'entrée aux services offerts par la catégorie. L'objectif de cette interface est de mieux contrôler les échanges d'une catégorie avec le reste du système et de faciliter le bon usage d'une catégorie.
5. Concevoir la structure de la présentation (maquettes et classes IHM)
6. Organiser la configuration logicielle : A priori, chaque catégorie se transforme en un sous-système de configuration logicielle. Il faut cependant prendre en compte les parties à extraire pour bénéficier de la réutilisation de code. Par exemple, le sous-système de présentation des commandes doit isoler spécifiquement un sous-système regroupant les classes nécessaires à la construction de la fenêtre de recherche et de sélection de commandes.

Conception détaillée

1. Concevoir le modèle logique (output : Modèle logique de conception détaillée)
   1. Concevoir les classes
   2. Concevoir les associations
   3. Concevoir les attributs
   4. Concevoir les opérations
   5. Valider le modèle
2. Développer la configuration logicielle (configuration logicielle détaillée)



2TUP et UML

Capture des besoins fonctionnels	• Diagramme des cas d'utilisation, • Diagrammes de séquence, • Diagrammes de collaboration
Analyse	• Diagramme de classes, • Diagrammes d'états transition
Capture des besoins techniques	• Diagramme des cas d'utilisation
Conception générique	• Diagramme de déploiement
Conception préliminaire	• Diagramme de composants, • Diagramme de déploiement
Conception détaillée	•Diagramme de classes, •Diagramme de séquence, •Diagramme de collaboration, •Diagramme d'états, •Diagramme d'activités, •Diagramme de composants